

# **Real-Time Airborne Object Detection and Tracking**

*Deep Learning Applications — (Assignment 3B) Final Project Report*

Aditya Ved

Student ID: a1989895

Master of Artificial Intelligence & Machine Learning

Adelaide University

## Table of Contents

### 0. Demo

### 1. Introduction

#### 1.1 Problem Statement

#### 1.2 Motivation

#### 1.3 Contributions

### 2. Related Work

#### 2.1 Gap This Project Addresses

### 3. Methodology

#### 3.1 System Overview

#### 3.2 Detector Architecture

#### 3.3 Loss Functions

#### 3.4 Tracking

#### 3.5 Depth Direction (Approaching / Receding / Stable)

#### 3.6 Rough Metric Distance Estimate

### 4. Experimental Setup

#### 4.1 Dataset

#### 4.2 Object Size Analysis

#### 4.3 Data Quality Checks and Preprocessing

#### 4.4 Hardware and Software

#### 4.5 Hyperparameters

## **5. Results**

### **5.1 Test-Set Performance (Overall)**

### **5.2 Per-Class Performance**

### **5.3 Training Curves**

### **5.4 Confusion Matrix**

### **5.5 Note on Baseline Comparison**

### **5.6 Qualitative Results on Real Video**

## **6. Discussion**

### **6.1 Obstacles and How They Were Overcome**

#### **6.1.1 Corrupt segmentation-format labels**

#### **6.1.2 Mid-training crash**

### **6.2 What I Learned**

### **6.3 Future Directions**

## **7. Conclusion**

## **8. References**

## 0. Demo

Demo Video to intuitively highlight what the project achieves : <https://youtu.be/e2RxPrOq2vU>

## 1. Introduction

### 1.1 Problem Statement

Aeroplanes, birds, drones, and helicopters share the same airspace and often look alike at distance. This project builds a system that detects, classifies, and tracks these four classes in real time on consumer hardware, with stable per-object identity preserved across video frames. The deliverable is a single pipeline that goes from a raw video frame to an annotated frame containing class labels, confidence scores, persistent track IDs, per-frame class counts, a depth direction indicator (approaching, receding, or stable), and a rough metric distance estimate.

### 1.2 Motivation

Counter-UAS, bird-strike monitoring near runways, airspace situational awareness, and security surveillance all need the same three things at once: low latency, stable identities across frames, and the ability to run without a server-class GPU. Two-stage and transformer-based detectors are accurate but heavy. The interesting question is not whether a real-time detector can recognise these four classes at all, but whether one can be tuned well enough that the rest of the pipeline (tracking, depth, distance) becomes useful information rather than noise. This project tests that.

### 1.3 Contributions

- Fine-tuned a YOLOv26s detector on a 22.5K-image airborne dataset, reaching 0.974 mAP@50 and 0.715 mAP@50-95 on the held-out test split, with per-class precision and recall above 0.94 on all four classes.
- Built a real-time tracking pipeline on top of the detector that combines BoTSORT for persistent IDs, a log-linear bbox-area-rate model for depth direction, and a pinhole-camera model for a rough metric distance estimate.
- Stress-tested the model on out-of-distribution video and documented exactly where it fails (specific lighting and distance regimes), turning the result into a concrete data-collection recommendation.

## 2. Related Work

Airborne object detection sits at the intersection of three families of detectors. Two-stage detectors like Faster R-CNN and Cascade R-CNN first propose regions then classify them, which is accurate but inherently slower than single-stage alternatives and requires manual anchor tuning. Get the anchor scales wrong for small objects like drones and recall collapses. Transformer detectors like RT-DETR, DINO, and Co-DETR replace anchors with set prediction and Hungarian matching, but their attention computation scales quadratically with token count and they need substantial GPU memory and post-processing complexity that does not fit a real-time, consumer-hardware target.

Single-stage YOLO-family detectors take the opposite trade-off: one forward pass produces all predictions, no region proposals, anchor-free in Yolo26. It is purpose-built for edge and low-power

devices, replaces NMS post-processing with a one-to-one detection head, and is specifically designed to handle small objects through its STAL (Small Target Assignment Learning) and ProgLoss mechanisms.

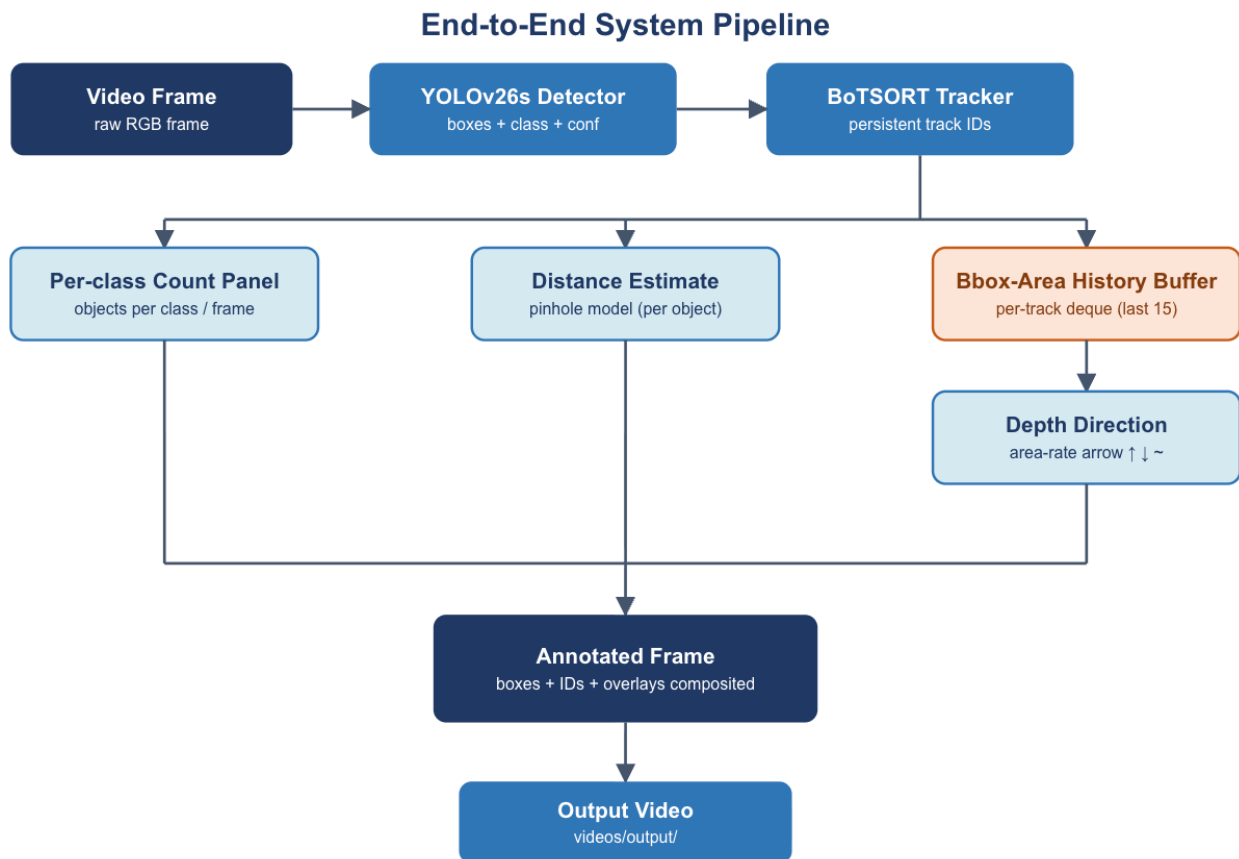
## 2.1 Gap This Project Addresses

A clean fine-tuning of a recent real-time detector on this specific four-class airborne dataset has not been published in an end-to-end deployable form. More importantly, the academic detection literature stops at mAP numbers. What a downstream operator actually needs (persistent track IDs across frames, a depth direction signal, an order-of-magnitude distance estimate) is rarely shipped alongside the detector. This project closes that gap on a single consumer machine.

## 3. Methodology

### 3.1 System Overview

The pipeline is split into two stages. Stage one is a fine-tuned YOLOv26s detector that takes a single frame and outputs bounding boxes with class labels and confidence scores. Stage two consumes those detections, assigns persistent IDs using BoTSORT, accumulates short bounding-box area histories per track, and derives the per-frame display features (count panel, depth direction, distance estimate) from those histories.



**Figure 1.** End-to-End System Pipeline

### 3.2 Detector Architecture

YOLOv26s has three architectural components, all of which are fine-tuned in this project:

- **Backbone** — the feature extractor. It downsamples the input image through convolutions and C3k2 blocks, producing feature maps at multiple scales (P2 through P5). Deeper layers carry richer semantic information, shallower layers carry finer spatial detail.
- **Neck** — a PAN/FPN-style feature fusion network. It combines features from different backbone depths using upsampling, concatenation, and more C3k2 blocks, so that every detection scale has access to both spatial detail and semantic context.
- **Head** — the prediction module. It outputs detections at three scales (P3, P4, P5 — corresponding to 80×80, 40×40, and 20×20 grids on a 640px input) using a one-to-one head that removes the need for non-maximum suppression at inference time.

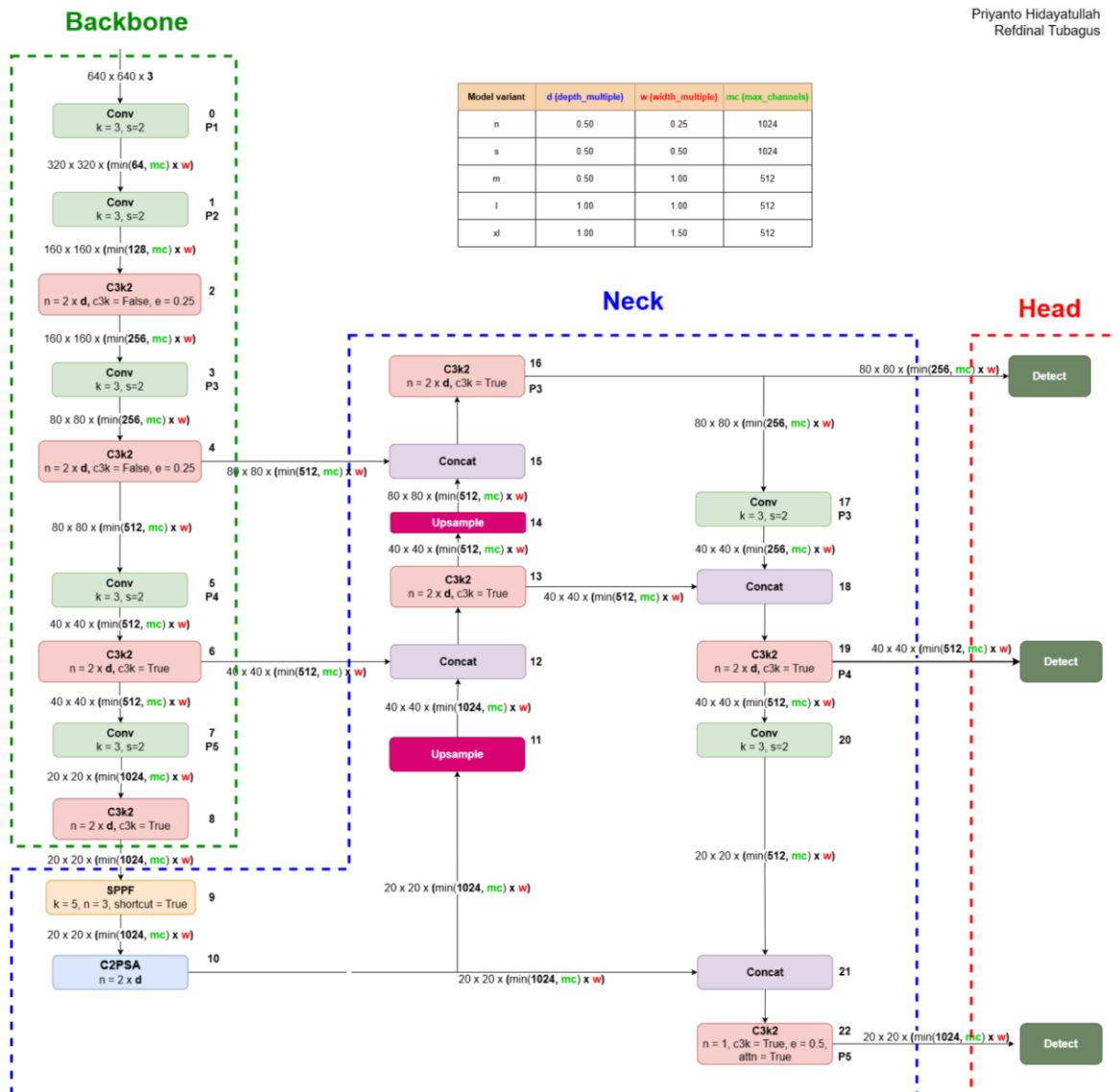


Figure 2. Yolo26 Architecture

*Detector architecture: Input (640×640×3) → Backbone (multi-scale features) → Neck (PAN/FPN fusion) → Head (three-scale predictions) → final boxes + classes.*

### 3.3 Loss Functions

YOLOv26 training optimises three losses simultaneously:

- **Box loss ( $L_{\text{box}}$ )** — a CloU-style loss penalising geometric mismatch between predicted and ground-truth boxes. It captures center distance, overlap, and aspect-ratio mismatch in one term.

$$L_{\text{box}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

where:

- $\text{IoU}$  = intersection-over-union between predicted box  $\mathbf{b}$  and ground-truth box  $\mathbf{b}^{gt}$
  - $\rho(\mathbf{b}, \mathbf{b}^{gt})$  = Euclidean distance between the two box centers
  - $c$  = diagonal length of the smallest enclosing box containing both
  - $v = \frac{4}{\pi^2} \left( \arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2$  (aspect-ratio consistency)
  - $\alpha = \frac{v}{(1 - \text{IoU}) + v}$  (trade-off weight)
- **Classification loss ( $L_{\text{cls}}$ )** — a binary cross-entropy term over the four object classes, measuring how well each predicted box gets the class right.

$$L_{\text{cls}} = - \sum_{c=1}^C \left[ y_c \log(\hat{p}_c) + (1 - y_c) \log(1 - \hat{p}_c) \right]$$

where:

- $y_c \in \{0, 1\}$  — ground-truth indicator for class  $c$
- $\hat{p}_c = \sigma(z_c) = \frac{1}{1 + e^{-z_c}}$  — predicted probability (independent sigmoid per class, not softmax)

- **Distribution Focal Loss (L\_dfl)** — instead of predicting each box edge as a single point, the model predicts a probability distribution over discrete bins along that edge. L\_dfl trains those distributions to be sharp and correct, leading to tighter, more confident box edges.

$$L_{dfl} = - \left[ (y_{i+1} - y) \log(S_i) + (y - y_i) \log(S_{i+1}) \right]$$

where:

- $S_i = \frac{e^{z_i}}{\sum_{j=0}^n e^{z_j}}$  — softmax probability for bin  $i$
- Only the two bins bracketing  $y$  contribute (linear interpolation in log-space)
- Predicted edge offset is recovered as the expectation:  $\hat{y} = \sum_{i=0}^n S_i \cdot y_i$

The total training objective is a weighted sum of the three:

$$L_{total} = \lambda_{box} \cdot L_{box} + \lambda_{cls} \cdot L_{cls} + \lambda_{dfl} \cdot L_{dfl}$$

with the weights set to Ultralytics defaults:  $\lambda_{box} = 7.5$ ,  $\lambda_{cls} = 0.5$ ,  $\lambda_{dfl} = 1.5$

### 3.4 Tracking

BoTSORT is layered on top of the detector. For every new frame, BoTSORT matches incoming detections to existing tracks using a combination of motion prediction (Kalman filter) and appearance similarity. Each track is assigned a stable integer ID that persists across frames as long as the object remains visible (and through brief occlusions). This is what turns per-frame detections into objects with identities.

### 3.5 Depth Direction (Approaching / Receding / Stable)

For each tracked object, we maintain a sliding window of the last 15 bounding-box areas. We fit a straight line through the log of those areas against the frame index, multiply the slope by the frame rate, and convert to a percentage rate of area change per second:

$$rate\ (\%/s) = slope(\log(area\_t) \text{ vs. } t) \times fps \times 100$$

If the rate is above +5 %/s the object is labelled "approaching", below -5 %/s it is "receding", and within  $\pm 5$  %/s it is "stable". Fitting in log-space makes the rate scale-invariant, so an object that doubles in size is treated the same way regardless of starting size.

### 3.6 Rough Metric Distance Estimate

Using the pinhole-camera model, the distance from the camera to an object whose largest visible dimension is `bbox_size` pixels is:

$$distance\_m = (real\_size\_m \times focal\_length\_px) / bbox\_size\_px$$



Each class is given a midpoint real-world size (aeroplane 35 m wingspan, bird 1 m wingspan, drone 0.5 m diagonal, helicopter 14 m rotor diameter), and the focal length is set to 950 px (a reasonable default for a typical smartphone camera capturing at  $\sim 1080$ px width). This produces an order-of-magnitude estimate, not precision ranging. The two main sources of error are unknown aircraft subclass (a Cessna and a Boeing 737 share the "aeroplane" label but have very different wingspans) and an unknown true focal length.

## 4. Experimental Setup

### 4.1 Dataset

The dataset is the publicly available Airborne Object Detection dataset on Mendeley Data (*cd5z895tr2*), containing 22,470 training images, 6,494 validation images, and 3,227 test images, all at  $1024 \times 1024$  resolution. Labels are in YOLO format (class, x\_center, y\_center, width, height). The four classes are aeroplane, bird, drone, and helicopter.

#### Dataset composition

| Class        | Train         | Valid        | Test         | Total         |
|--------------|---------------|--------------|--------------|---------------|
| aeroplane    | 5,505         | 1,625        | 767          | 7,897         |
| bird         | 5,522         | 1,557        | 821          | 7,900         |
| drone        | 5,500         | 1,602        | 796          | 7,898         |
| helicopter   | 5,528         | 1,585        | 787          | 7,900         |
| no object    | 415           | 125          | 56           | 596           |
| <b>TOTAL</b> | <b>22,470</b> | <b>6,494</b> | <b>3,227</b> | <b>32,191</b> |

**Table 1.** Annotation counts per class per split. Annotations are well-balanced across classes in every split.

The image counts in the splits are skewed because a single image can contain multiple annotated objects; what matters for training is annotation balance, which the table above confirms.

### 4.2 Object Size Analysis

Object size in pixels because small objects carry less signal. Median bbox areas per class on the training split are:

| Class     | Median area (norm.) | At 640px input<br>( $\sim$ side)                  | At 1024px input<br>( $\sim$ side)                       |
|-----------|---------------------|---------------------------------------------------|---------------------------------------------------------|
| drone     | 0.00317             | $\sim 1,300 \text{ px}^2$ ( $\sim 36 \times 36$ ) | $\sim 3,328 \text{ px}^2$ ( $\sim 58 \times 58$ )       |
| bird      | 0.00767             | $\sim 3,141 \text{ px}^2$ ( $\sim 56 \times 56$ ) | $\sim 8,041 \text{ px}^2$ ( $\sim 90 \times 90$ )       |
| aeroplane | 0.01565             | $\sim 6,409 \text{ px}^2$ ( $\sim 80 \times 80$ ) | $\sim 16,408 \text{ px}^2$<br>( $\sim 128 \times 128$ ) |

| Class      | Median area (norm.) | At 640px input (~side)          | At 1024px input (~side)            |
|------------|---------------------|---------------------------------|------------------------------------|
| helicopter | 0.02016             | ~8,259 px <sup>2</sup> (~91×91) | ~21,144 px <sup>2</sup> (~145×145) |

**Table 2.** Median object size per class. Drone is the smallest class by a wide margin and sits just above the COCO small-object threshold (32×32) at 640px input.

### 4.3 Data Quality Checks and Preprocessing

Two verification scripts were run end-to-end on the dataset: `verify.py` checks image–label pairing, image readability, YOLO label format compliance, and reports class distribution; `validate_all_splits_dimensions.py` confirms every image is exactly 1024×1024. Both passed for the bulk of the dataset, but `verify.py` flagged three label files that contained polygon/segmentation-style annotations (21–27 values per line) instead of the expected YOLO bounding-box format (5 values per line). These three image/label pairs were removed since removing them doesn’t significantly affect the size of the dataset.

No additional preprocessing was applied. Images are resized to 640×640 at training time by Ultralytics. Mosaic augmentation is enabled by default and disabled for the final 10 epochs to give the model a cleaner convergence window.

### 4.4 Hardware and Software

| Resource            | Detail                                                            |
|---------------------|-------------------------------------------------------------------|
| Compute             | Apple M4 Max, 128 GB unified memory, MPS backend (with GPU)       |
| Framework           | Ultralytics 8.4.33, PyTorch 2.11.0, Python 3.11                   |
| Detector weights    | YOLOv26s, pretrained on COCO                                      |
| Tracker             | BoTSORT (Ultralytics built-in)                                    |
| Total training time | ~3.9 hours wall-clock across 100 epochs (with one mid-run resume) |

**Table 3.** Hardware and software stack used for training and evaluation.

### 4.5 Hyperparameters

| Hyperparameter            | Value                                          |
|---------------------------|------------------------------------------------|
| Image size (imgsz)        | 640                                            |
| Batch size                | 128                                            |
| Epochs                    | 100 (best checkpoint at epoch 99 by mAP@50-95) |
| Patience (early stopping) | 20 epochs without mAP improvement              |
| Dataset cache             | "ram" — full dataset preloaded into RAM        |

| Hyperparameter            | Value                                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------------|
| Mosaic augmentation       | Enabled, disabled for final 10 epochs (close_mosaic=10)                                                        |
| Mixed precision (AMP)     | Disabled (for compatibility with MPS backend)                                                                  |
| Optimizer / LR / schedule | Ultralytics defaults; learning rate decayed from $\sim 4.15\text{e-}4$ to $\sim 4.34\text{e-}5$ across the run |
| Checkpoint frequency      | Every epoch (save_period=1)                                                                                    |

**Table 4.** Training hyperparameters. All settings are reproducible from `train.py` and `train_resume.py` in the project repository.

## 5. Results

### 5.1 Test-Set Performance (Overall)

All numbers in this section come from running `model.val(split="test")` on the fine-tuned `best.pt` against the held-out 2,241-image test split, with the standard low-confidence threshold (0.001) used for mAP computation.

| Metric                 | Value         |
|------------------------|---------------|
| mAP@50                 | 0.9740        |
| mAP@50-95              | 0.7152        |
| Precision              | 0.9559        |
| Recall                 | 0.9413        |
| Images processed       | 2,241         |
| Images with detections | 2,169 (96.8%) |
| Total detections       | 3,274         |

**Table 5.** Overall test-set performance of the fine-tuned YOLOv26s model.

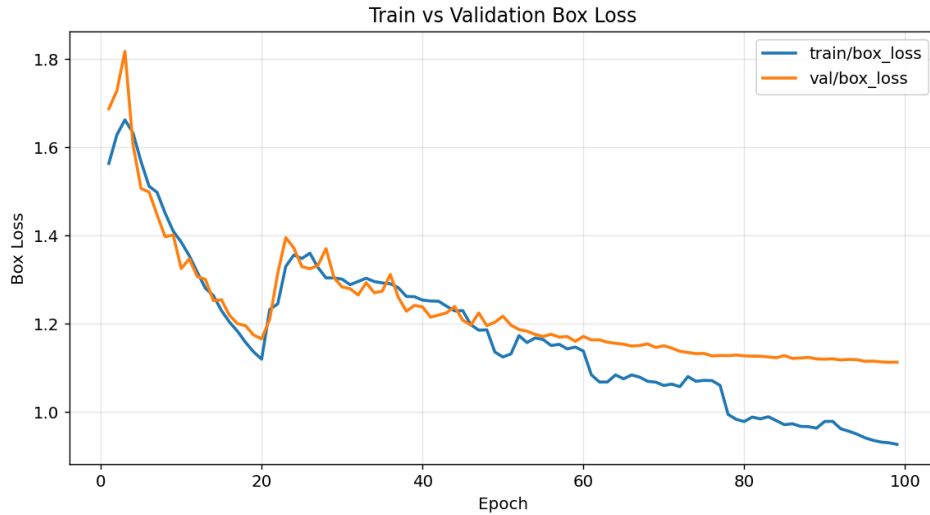
### 5.2 Per-Class Performance

| Class      | Precision | Recall | mAP@50 | mAP@50-95 |
|------------|-----------|--------|--------|-----------|
| aeroplane  | 0.9601    | 0.9557 | 0.9829 | 0.7585    |
| bird       | 0.9720    | 0.9549 | 0.9818 | 0.7611    |
| drone      | 0.9470    | 0.9045 | 0.9546 | 0.6609    |
| helicopter | 0.9444    | 0.9501 | 0.9767 | 0.6802    |

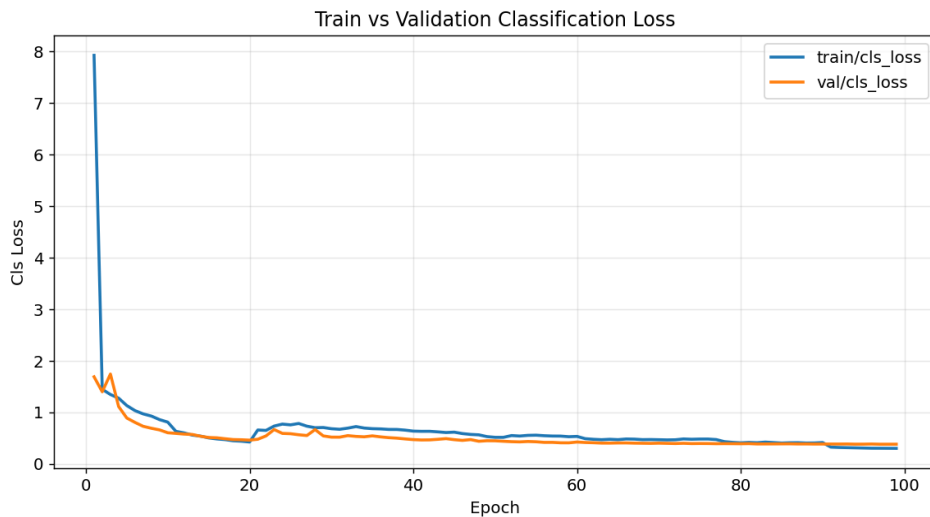
**Table 6.** Per-class performance on the test split. Aeroplane and bird score highest on mAP@50-95, drone the lowest.

Aeroplane and bird score best on mAP@50-95 because both are also COCO classes — the model started from features that already knew how to localise these objects, and fine-tuning only had to refine them. Drone and helicopter are not in COCO, so the model had to learn them from scratch on this dataset; drone is additionally hurt by being the smallest class on average (median  $\sim 36 \times 36$  px at 640px input).

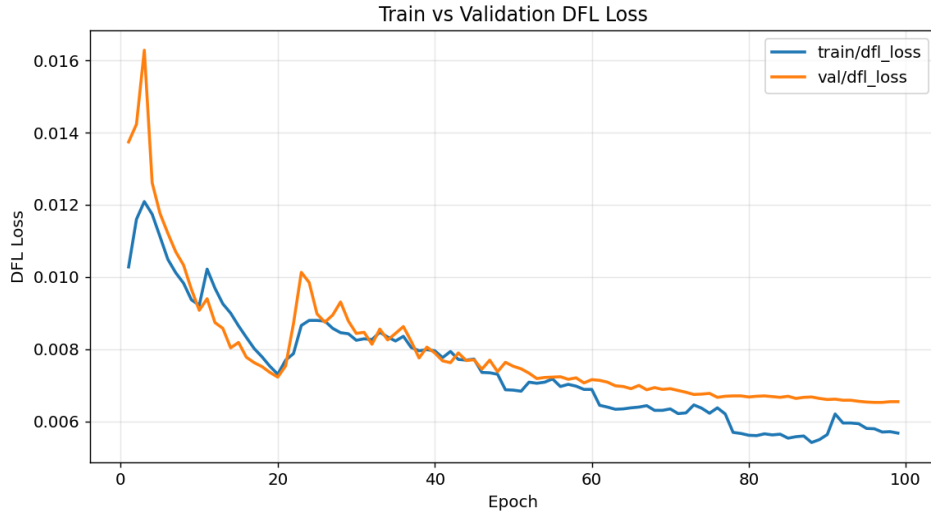
### 5.3 Training Curves



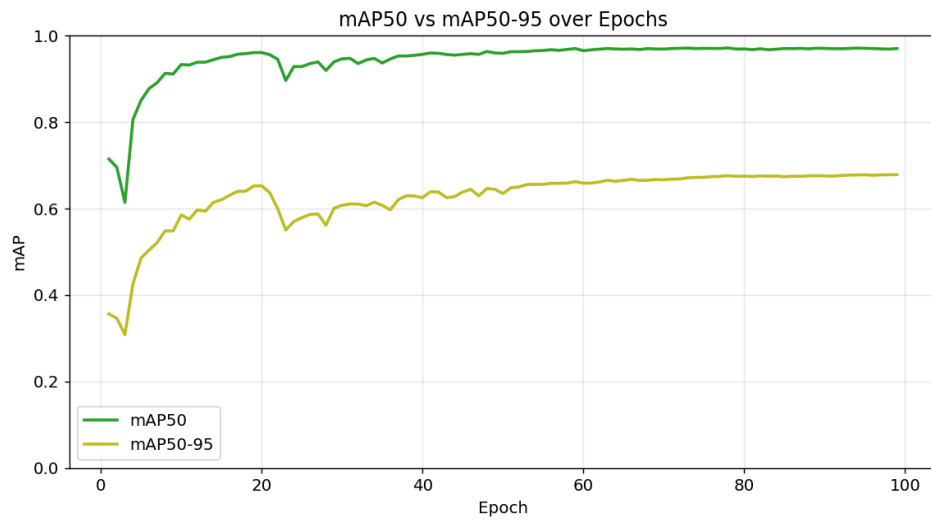
**Figure 3.** Train vs validation box loss across 100 epochs. Both fall together through epoch  $\sim 20$  (clean learning). A brief instability appears around epoch 22 from a training resume. From epoch  $\sim 60$  onwards train keeps falling while val flattens — a widening gap that indicates mild localization overfitting.



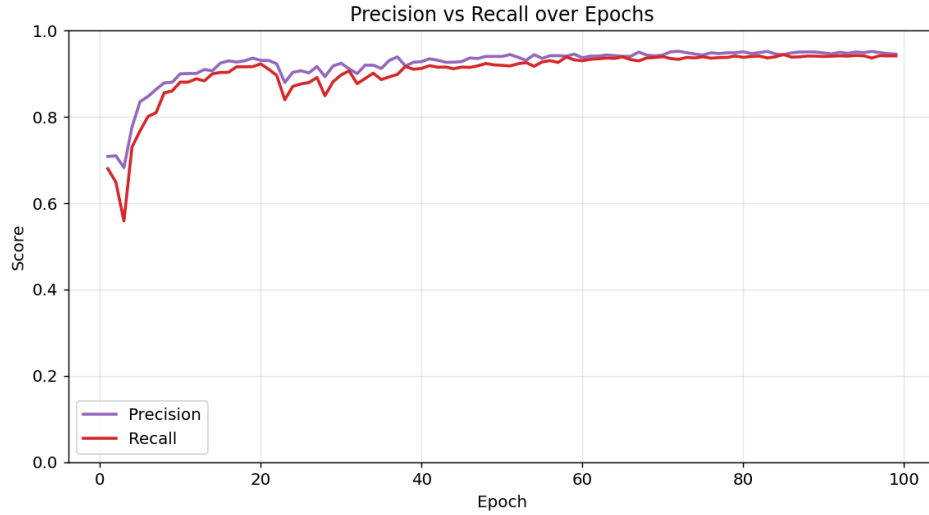
**Figure 4.** Train vs validation classification loss. The epoch-1 spike ( $\sim 7.9$ ) is random-init noise. From epoch 2 onwards train and val fall together cleanly.



**Figure 5.** Train vs validation DFL loss. Both decrease and stabilise together, with no significant divergence — the box-edge distributions generalise well.



**Figure 6.** mAP@50 vs mAP@50-95 over epochs. mAP@50 climbs quickly in epochs 1-5 and plateaus around 0.97. mAP@50-95 is consistently ~0.29 below mAP@50, indicating boxes find their objects but do not always hug them tightly.



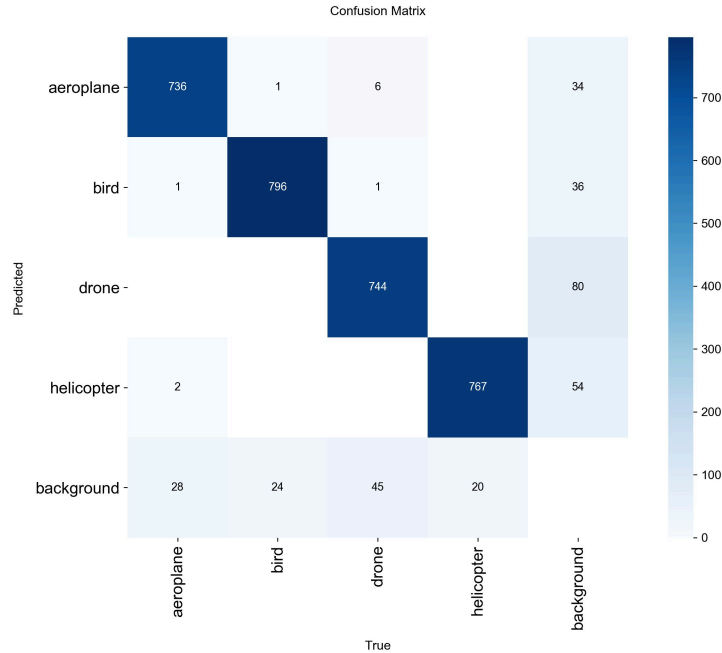
**Figure 7.** Precision and recall over epochs. Both rise together and end  $\sim 0.94$ . Precision sits slightly above recall throughout, indicating a healthy "confident before predicting" bias.

## 5.4 Confusion Matrix

The plots in this subsection were generated by Ultralytics directly from the test split during the validation run.



**Figure 8.** Normalized confusion matrix. Diagonal entries are above 0.93 for every class. The dominant source of error is background false positives, especially for drone (0.39 of background predicted as drone in the normalised view). Class-to-class confusion is negligible.



**Figure 9.** Raw confusion matrix. Each off-diagonal cell shows a single-digit or low-double-digit count out of ~800 instances per class — the model rarely confuses one airborne class for another.

## 5.5 Note on Baseline Comparison

A like-for-like comparison against an "out-of-the-box" YOLOv26 is not possible because drone and helicopter are not COCO classes — the un-fine-tuned model has no concept of them and would score zero on those two classes by construction. The fairest internal baseline is the first-epoch result in the same training run: precision 0.708, recall 0.680, mAP@50 0.715, mAP@50-95 0.356. Fine-tuning improved every one of those metrics by 24–36 absolute percentage points.

## 5.6 Qualitative Results on Real Video

To test how well the model generalises beyond the dataset distribution, it was run on several real-world videos of airborne objects. The pipeline performs well in most clips — boxes are stable, IDs persist, depth direction and distance estimates behave sensibly. Failure modes are also clear: certain combinations of lighting and object distance (very small objects against bright sky, or low-contrast scenes at dusk) produce missed detections. These conditions are under-represented in the training set, — it points at the kind of camera-specific data that would be needed for production deployment.

# 6. Discussion

## 6.1 Obstacles and How They Were Overcome

### 6.1.1 Corrupt segmentation-format labels

Initial verification with `verify.py` flagged three label files containing polygon-style annotations (21–27 values per line) instead of YOLO bounding-box format (5 values per line). With 22,470 training images,

hand-converting three was not worth the engineering time; both the images and their labels were removed. It saved a crash much later.

### 6.1.2 Mid-training crash

Training was interrupted partway through the first 100-epoch run. Ultralytics writes `last.pt` every epoch (`save_period=1` in `train.py`), so `train_resume.py` was able to restart with `resume=True`, restoring optimiser state, scheduler position, and epoch counter from the checkpoint. The patience parameter was bumped from 5 to 20 on resume to give the model more headroom against the early-stopping criterion. The visible scar from this is the brief instability bump in all three loss curves and the mAP curve around epoch 22 — the model recovered within a few epochs.

## 6.2 What I Learned

- Object size in pixels matters more than image count. Drone has roughly as many training annotations as aeroplane, but drone is the worst-performing class because it is intrinsically smaller. No amount of additional drone images at the current resolution would close the entire gap — moving to a larger input size (`imgsz=1024`) would help more than tripling the drone count.
- Fine-tuning transfers strongly within the natural-image domain. Aeroplane and bird are COCO classes; their `mAP@50-95` finished well above the two non-COCO classes. The model was not learning aeroplane from scratch, only re-aiming features it already had.
- `cache="ram"` is a huge throughput win on the M4 Max. Preloading the ~19 GB decoded dataset into RAM means the disk is touched once, not once per epoch. On a machine with 128 GB of unified memory and an MPS backend, this is the difference between epoch times being IO-bound and being compute-bound.
- `mAP@50` on its own is misleading. A model can sit at 0.97 `mAP@50` and still produce boxes that are visibly loose around objects. `mAP@50-95` is the more honest scoreboard; reporting both lets a reader spot localisation-versus-recognition issues immediately.
- Real-world video is a different evaluation set. The test split shares its distribution with train and val. Production video does not. Any project that stops at test-split metrics has not actually been evaluated.
- Detection on its own is not enough. Adding tracking, depth direction, and distance turns four numbers per object into a usable situational picture. The added components are cheap compared to the detector, and they make the same detector dramatically more useful downstream.

## 6.3 Future Directions

- Train at `imgsz=1024` instead of 640. The dataset is natively 1024×1024, so no information is added — but at 1024px input the median drone occupies ~3,328 px<sup>2</sup> (~58×58) instead of ~1,300 px<sup>2</sup> (~36×36). This is the single most direct lever to close the drone performance gap.
- Collect deployment-specific training data. The current model should be treated as a strong starting point, not a final product. Fine-tuning further on images captured from the actual deployment camera position, in the actual weather and lighting conditions, is the standard recipe for closing the gap exposed by the video test.
- Subclass aircraft to remove the biggest source of distance error. The current pinhole-model distance estimate assumes a single midpoint size per class — Cessna and Boeing 737 share the



"aeroplane" label but their wingspans differ by  $\sim 3\times$ . Fine-tuning a second-stage classifier to predict aircraft subclass would let the distance estimator pick a much tighter assumed size and bring the error from  $\pm 300\%$  down to perhaps  $\pm 50\%$ .

## 7. Conclusion

This project delivered a working real-time airborne object detection and tracking system on consumer hardware. A YOLOv26s detector fine-tuned on a 22.5K-image, four-class airborne dataset reached 0.974 mAP@50 and 0.715 mAP@50-95 on the held-out test split, with per-class precision and recall above 0.94 on every class. On top of that detector, a BoTSORT tracker maintains persistent object identities across frames, a log-linear bbox-area model produces a depth direction signal, and a pinhole-camera model produces an order-of-magnitude metric distance estimate — together giving a per-object output that is substantially richer than detection alone.

The honest result is that the model is strong on the dataset distribution and weak on parts of the real world the dataset does not cover. The two clearest paths to a deployable system are: (a) increase the input resolution to give the smallest class (drone) more pixels to work with, and (b) collect environment-specific data from the actual deployment camera and fine-tune again.

## 8. References

- [1] Airborne Object Detection dataset, Mendeley Data, doi:10.17632/cd5z895tr2.1. <https://data.mendeley.com/datasets/cd5z895tr2/1>
- [2] Ultralytics YOLO documentation and source code (YOLOv26, BoTSORT integration, training API). <https://docs.ultralytics.com>
- [3] Aharon, N., Orfaig, R., Bobrovsky, B.-Z. "BoT-SORT: Robust Associations Multi-Pedestrian Tracking", 2022. <https://arxiv.org/abs/2206.14651>
- [4] Li, X., Wang, W., Wu, L. et al. "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection" (DFL), NeurIPS 2020. <https://arxiv.org/abs/2006.04388>
- [5] Zheng, Z. et al. "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression" (CloU), AAAI 2020. <https://arxiv.org/abs/1911.08287>
- [6] Lin, T.-Y. et al. "Microsoft COCO: Common Objects in Context", ECCV 2014. (Source of aeroplane and bird pretrained classes.) <https://arxiv.org/abs/1405.0312>
- [7] Project source code and notebooks, AdityaVed95/Realtime-Airborne-Object-Detection-and-Tracking. <https://github.com/AdityaVed95/Realtime-Airborne-Object-Detection-and-Tracking>